K215: Smaller Proofs and Quotient Detachment

Research note prepared for the ProveIt project

21 October 2026

Abstract

We give a further simplification of the currently checked 215-state construction and a new freedom in its numerical inference system. First, dominated implication can recover unrestricted implication through pointwise lifting, without an auxiliary deduction theorem or its distribution lemma. The replacement Lean module is 26,709 bytes, compared with the current 30,114 bytes; its unchanged downstream calculus-equivalence theorem has been freshly checked. Second, modus ponens can ignore the two constructor bits of its major premise, provided soundness is maintained as validity under every valuation in an irreflexive model with two distinct elements. This statement is checked against the repository's actual numerical decoding, and both restrictions have exact checked counterexamples. The enlarged calculus is formally proved equivalent to the repository's ZFC-refutation predicate, with all 110 modules freshly compiled. The enlarged rule preserves the relevant contracts of the original completeness compiler. Its first physical implementation, however, increases source and allocation costs. No verified 214-state machine is obtained. We identify the existing conditional-marker CM215 as an economical additional target for proof transport, distinguish incoming work from this contribution, and provide source, patches, checks, and verification records.

Contents

1 Results and exact boundaries **2**

2 The current baseline already incorporates substantial simplification **2**

3 Unrestricted implication by pointwise lifting **3**

4 Quotient detachment **5**

5 Why the existing completeness compiler still works **6**

6 Physical implementation and its measured cost **7**

7 The easiest additional 215-state proof **8**

8 The scoped search for 214 states **8**

9 Recommended integration order **9**

10 Archive and reproduction **9**

1 Results and exact boundaries

The requested endpoint is

$$\mathrm{ZFC} \vdash \neg\,\mathrm{Halt}(M, \mathrm{blank}) \longleftrightarrow \mathrm{Con}(\mathrm{ZFC}).$$

We use one two-way infinite tape, an initially all-zero tape, the head at position zero, two symbols, and left/right moves only. Working-state counts include initialization and exclude the separate halt. The accepted K215 has 215 working states and 430 transition rows. Its dense table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090.`

The repository audit is pinned to `53269bcab1efc0f82d3022645bd7cf0c22d84959`. Live sources were read through the GitHub connector; local copies were used as build inputs with explicit source hashes.

Result	Status
Shorter implication proof	Fresh Lean compilation, including unchanged `KCodeDom.cdom_iff_c222`; exact generator reproduction and integration patch.
Rocq-style companion	Direct proof compiled under Coq 8.18.0; independently accepted by recursive `coqchk`. This is an alternative to the smaller reflected production file.
Quotient detachment	Fresh Lean check using actual `Code254` constructors and satisfaction lemmas; actual sharp counterexamples also checked.
C222 extension soundness	Fresh check against the existing native-bundle soundness interface.
Full quotient calibration	Exact Lean theorem `Q222 F` iff `BProv ZFCax_s [] Form.fBot`; all 110 modules freshly compiled.
Additional 215-state target	Existing CM215, recommended for its small relative proof; not a newly discovered table.
214-state construction	None established. Source experiments and bounded negative searches are included.

A proof-assistant theorem about finite ZFC refutations is not automatically an encoded internal PA derivation. The repository explicitly distinguishes these endpoints. The local refactor preserves the existing calculus interface, but we do not claim a fresh rebuild of the complete 185-module binary-machine endpoint or completion of the separate internal-arithmetic milestone. The quotient rule is checked through the exact numerical-calculus-to-ZFC-refutation equivalence. A physical machine using it still needs its own source binding, quantified runtime, and startup proofs.

2 The current baseline already incorporates substantial simplification

Both the main branch and the incoming directory were inspected at the pinned revision. Production already shares the dominated-implication argument, derives reverse source simulation by symmetry, executes enumerator blocks symbolically, composes AX from macro contracts, and cancels executed prefixes. The primary Lean closure has already shrunk from 204 to 185 project modules [2, 3, 5].

The current implication file is 30,114 bytes, rather than the earlier 689,857-byte expanded development. Our comparison is against the current file: 30,114 to 26,709 bytes, a saving of 3,405 bytes or about 11.31%. The much larger earlier reduction belongs to previous work.

Incoming packets also contain a normalized source relation with 1,426 checkpoints instead of 2,240 and a complete checked concrete Lean source equivalence. They contain checked CM215 transfer proofs, a compact ordinary Rocq source proof, and M124 all-tape equivalence proofs [8]. Thus trailing-zero normalization and generic positive stuttering are useful integration opportunities, but are not presented here as new discoveries.

The practical lesson is to simplify an exact interface and then rebuild its typed clients. A short delta proof retains its inherited calibration. A small new theorem does not erase the foundational work it imports. Conversely, a large finite certificate should not be retained merely because it was once the only implementation of an otherwise simple invariant.

3 Unrestricted implication by pointwise lifting

3.1 The numerical target

Write $a \Rightarrow b$ for the code constructor `wim a b`, not for meta-level implication. Let $T : \mathbb{N} \to \mathrm{Prop}$ satisfy truth, numerical modus ponens, and the dominated row

$$u \le a \quad \Longrightarrow \quad T(((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c))). \tag{1}$$

The `DomImp` structure is unchanged. The goal is the same formula for arbitrary a, b, c, u, without the numerical guard. It concerns arbitrary codes and assumes no formula-injectivity property, well-formedness test, consistency assertion, or model of ZFC.

The old proof derived composition and distribution, introduced an inductive proof relation under hypotheses, proved deduction, and discharged three hypotheses. We replace its last stage by a direct implication calculation.

3.2 The lifting lemma

Two retained schemas, `cd99` and `cd88`, give

$$T((A \Rightarrow B) \Rightarrow (A \Rightarrow (X \Rightarrow B))), \tag{2}$$
$$T((A \Rightarrow (X \Rightarrow B)) \Rightarrow ((X \Rightarrow A) \Rightarrow (X \Rightarrow B))). \tag{3}$$

Lemma 3.1 (Pointwise lifting)**.** *If $T(A \Rightarrow B)$, then $T((X \Rightarrow A) \Rightarrow (X \Rightarrow B))$ for every code X.*

Proof. Apply numerical modus ponens to (2) and the hypothesis, then apply it to (3) and the result. □

The orientation matters. Existing composition is

$$(A \Rightarrow B) \Rightarrow ((B \Rightarrow C) \Rightarrow (A \Rightarrow C)).$$

A known $B \Rightarrow C$ cannot simply be used as its first premise. The checked lifting lemma uses the two valid schemas above and does not rely on that tempting but invalid shortcut.

3.3 The final calculation

The bootstrap supplies

$$K(A, B) = A \Rightarrow (B \Rightarrow A),$$
$$J(A, B, U) = (A \Rightarrow B) \Rightarrow (((A \Rightarrow U) \Rightarrow A) \Rightarrow B),$$

and retained `cd91` supplies composition. Put

$$q = (c \Rightarrow u) \Rightarrow c, \quad f = (c \Rightarrow u) \Rightarrow a, \quad g = a \Rightarrow c, \quad d = b \Rightarrow c.$$

From $J(c, d, u)$ and $K(c, b)$, modus ponens gives $T(q \Rightarrow d)$. Lift twice, first under g, then under f:

$$T((g \Rightarrow q) \Rightarrow (g \Rightarrow d)),$$

$$T((f \Rightarrow (g \Rightarrow q)) \Rightarrow (f \Rightarrow (g \Rightarrow d))).$$

Composition at $c \Rightarrow u, a, c$ gives $T(f \Rightarrow (g \Rightarrow q))$. One final modus ponens yields $T(f \Rightarrow (g \Rightarrow d))$, exactly unrestricted `rho8`. Once the retained schemas are available, this last argument uses six modus-ponens applications, including the two lifts.

Theorem 3.2 (Unchanged endpoint)**.** *For every `DomImp T` and every $a, b, c, u \in \mathbb{N}$, the new module proves*

$$T(\mathrm{rho8}(a, b, c, u)).$$

3.4 Concrete proof and build savings

The replacement removes `cd100`–`cd106`, the auxiliary `Proof` datatype, its soundness induction, and the discharge lemmas. It retains 58 bootstrap schemas and 52 CD schemas, compared with 59 CD schemas before. The new helper is `lift_right`; `distribution` is unnecessary in the checked endpoint lane. The public `rho8_all` statement is unchanged, and the unchanged `cdom_iff_c222` compiles.

Removing the ancillary public helper `distribution` is an API change. A separate consumer that needs it should keep a compatibility module or migrate its use. Retaining the whole deduction-theorem construction in the principal calibration file is unnecessary for the inspected endpoint.

The fresh thirteen-module build includes the exact reduced file and its calculus adapter. The new implication and adapter theorems report only `propext` and `Quot.sound`. The archive records compiler version, commands, source hashes, logs, and an integration patch. This is a local proof rebuild, not a new check of the complete binary-to-ZFC chain.

A direct Coq companion compiled under Coq 8.18.0 and passed an independent recursive `coqchk`. Its `rho8_all` assumption audit is closed under the global context. Its 24,352-byte, 440-line source is larger in bytes than the current 22,673-byte, 1,169-line reflected Rocq certificate. It offers the same direct proof structure without the reflection interpreter and 735-instruction table; it is not a claimed Rocq byte reduction. The two imported numerical modules match the pinned upstream Git blobs without edits. This independent check used Coq 8.18.0 rather than the full endpoint's Rocq 9.1.0 toolchain.

The source generator also needed a reproducibility repair. Its pinned CD template had different hashes under LF and CRLF line endings. The updated generator normalizes only CRLF to LF, verifies the canonical hash, and emits the exact checked module. This does not weaken the mathematical checker or allow arbitrary templates. The regeneration receipt proves byte-for-byte reproduction of the delivered Lean source.

4 Quotient detachment

4.1 The unchanged code algebra

The actual repository constructors are [4]

$$\begin{aligned} \mathrm{P}(a,b) &= \frac{(a+b)(a+b+1)}{2} + b, \\ I(a,b) &= 4\,\mathrm{P}(a,b), & M(a,b) &= 4\,\mathrm{P}(a,b) + 1, \\ Q(a,b) &= 4\,\mathrm{P}(a,b) + 2, & E(a,b) &= 4\,\mathrm{P}(a,b) + 3. \end{aligned}$$

They represent implication, membership, universal quantification, and equality. In $Q(a,b)$, a is a variable index and b a formula code. Truth is code zero, including $I(0,0) = 0$. In an irreflexive membership structure, code $1 = M(0,0)$ is false.

Let a structure D contain distinct elements $x \neq y$, and let R be irreflexive. Define

$$\mathrm{Valid}_R(n) \quad \Longleftrightarrow \quad \forall e : \mathbb{N} \to D,\ \mathrm{Sat}_R(e, \mathrm{dec}(n)).$$

This is validity in one structure under every valuation.

Theorem 4.1 (Quotient detachment)**.** *For every $a, b, h \in \mathbb{N}$,*

$$\mathrm{Valid}_R(a) \wedge \mathrm{Valid}_R(h) \wedge \lfloor h/4 \rfloor = \mathrm{P}(a,b) \quad \Longrightarrow \quad \mathrm{Valid}_R(b).$$

Proof. Write $h = 4\,\mathrm{P}(a,b) + r$, $0 \leq r < 4$.

If $r = 0$, then $h = I(a,b)$ and the repository's `sat_wim` law gives modus ponens, including the zero alias.

If $r = 1$, then $h = M(a,b)$. At the constant assignment with value x, validity would imply $R(x,x)$, contradicting irreflexivity. Thus this valid major premise is impossible.

If $r = 2$, then $h = Q(a,b)$. At any valuation e, instantiate the quantifier with $e(a)$. Updating e at a with its existing value leaves e unchanged, proving the interpretation of b.

If $r = 3$, then $h = E(a,b)$. If the numerical indices a and b differ, assign distinct elements to them, contradicting validity. Hence $a = b$ numerically, and the minor premise is the desired conclusion. □

The checked theorem uses the actual `Code254.dec` and its constructor-satisfaction laws. In particular, the equality case does not infer equality of indices from equality at a single valuation.

4.2 Exact limits of the theorem

Both hypotheses are sharp. In a singleton empty-membership structure, let

$$a = 0, \qquad b = 1, \qquad h = E(0,1) = 11.$$

Codes 0 and 11 are valid, while code 1 is false, although $11 \operatorname{div} 4 = 2 = \mathrm{P}(0,1)$. Therefore nonemptiness does not suffice. In a two-element empty-membership structure, the same codes give a counterexample at a constant valuation, so truth at one valuation does not suffice either. Both counterexamples are independent checked Lean theorems.

Different residues use the paired naturals differently: as formula codes, variable labels, or a binder and formula. A purely propositional informal interpretation would miss this. The complete semantic case split is what licenses the relaxed guard.

4.3 An equiconsistent enlarged system

Retain the accepted dominated calculus's truth, seed schemes, and theorem-only generalization. Replace numerical MP by

$$\frac{a \qquad h}{b} \qquad \text{if } h \operatorname{div} 4 = \mathrm{P}(a, b). \tag{QMP}$$

Every original MP inference is preserved. Consequently every old refutation remains a refutation in the enlarged system.

Conversely, the inherited consistency-to-model argument supplies an appropriate model satisfying the native seeds. A ZFC model, and the constructible model used by the repository's upper proof, is irreflexive and has two distinct elements. Unchanged seed soundness, theorem generalization, and quotient detachment preserve validity under all valuations. Code 1 is false. This gives the converse consistency implication and, by classical contraposition and the inherited calibration, the ordinary ZFC-formalizable equivalence of finite refutations. Equality of theorem sets is not required.

For direct formal reuse, `Quotient222.lean` defines a finite-derivation closure of `C222` under QMP and generalization. The base constructors expand into finite original derivations; no oracle is introduced. The file proves soundness under the exact native-bundle model hypothesis, rules out the falsity code there, and embeds every original refutation.

The four supplied modules finish this argument as an exact Lean theorem:

```
Quotient222.Q222 Code254.F ⟷
BProv ZFCax_s [] Form.fBot.
```

The upper adapter uses `KUpper222.bundles_in_L`; the empty set and its successor supply distinct elements of the constructible interpretation. The lower adapter reuses the short mathematical argument from `KEndpoint.c222_of_zf_refutation`, followed by `Q222.base`. It imports `KLowerQHyp` directly, avoiding unrelated tape-machine endpoints. All 110 modules were compiled afresh under Lean 4.32.0, with no cached project proof objects: 106 upstream modules match the pinned Git blobs and four new modules total 244 lines including comments and blanks. The final axiom report is exactly `propext`, `Classical.choice`, and `Quot.sound`; there is no added consistency axiom. This is refutational equivalence, not equality of all theorem sets, and it does not itself establish a new binary-machine endpoint.

5 Why the existing completeness compiler still works

The working tuple is (t, w, d, h): theorem, stack, counter, cache. For MP's field x, put $b = d + x$. The original command tests $h = 4\,\mathrm{P}(t, b)$. Success returns

$$(b, \mathrm{P}(0, w), b, 0),$$

and failure returns $(0, w, b, 0)$. Replace only the guard by $h \operatorname{div} 4 = \mathrm{P}(t, b)$; retain tags, arities, stack effects, returned counter, and cleanup.

An arbitrary run can accept additional commands. Nevertheless, all MP invocations used by the inherited strong completeness compiler have one of the following forms [5]:

Use	Data	Both guards
Actual inference	$h = 4\,\mathrm{P}(t, b)$	Succeed
Discard/reset	$h = 0$, $b = 1$	Fail: $\mathrm{P}(t, 1) > 0$
Positive counter loading	$t = h = 0$, $b = u > 0$	Fail: $\mathrm{P}(0, u) > 0$
Zero counter loading	Empty block	No invocation

The discard contract starts with cleared counter and cache, not an arbitrary dirty cache. A structural induction on the compiler transfers its command contracts. The cofinal candidate family and strictly draining reader use the unchanged command encoding, so the existing completeness route remains available.

For soundness, use validity under every valuation for the theorem, cache, and each stack entry in a fixed nontrivial model. DROP and NIP move valid values. AX supplies valid seeds and quantified formulas. MP failure inserts truth; success uses QMP. The alarm $h = 1$ contradicts the invariant.

This identifies the remaining obligations for a smaller machine: implement the quotient guard with exact normal-return registers, bind the entire source, prove its physical runtime and startup, and connect this semantic enumerator argument. The ordinary compiler-transfer proof is all-natural, but it is not a new binary-machine endpoint.

6 Physical implementation and its measured cost

A useful correction to the initial intuition is that K215 has no separate literal constructor-tag test to delete. Its MP routine constructs the tagged implication and tests numerical equality. Ignoring a tag therefore requires a different arithmetic routine.

At the selected source boundary, the inherited pairing computation has

$$R_1 = 0, \quad R_5 = h, \quad R_6 = 0, \quad R_7 = \mathrm{P}(a, b).$$

The proposed nine-primitive splice v00 consists of

$$\begin{aligned} D_4 &= (\mathrm{CD5}, ((\mathrm{CD5}, (\mathrm{CD5}, (\mathrm{CD5}, \mathrm{CI6}))), \mathrm{CC2})), \\ X &= (\mathrm{CD7}, (\mathrm{CI5}, \mathrm{CC2})), \end{aligned}$$

with replacement tree (D_4, X). It returns

$$R_1 = 0, \quad R_5 = \mathrm{P}(a, b), \quad R_6 = \lfloor h/4 \rfloor, \quad R_7 = 0.$$

The inherited equality continuation now implements the relaxed guard.

Each complete division iteration removes four cache units and adds one quotient unit. Its four terminal cases have durations $1, 4, 5, 6$; a complete group of four costs six source transitions. The subsequent transfer takes $3\,\mathrm{P}(a, b) + 1$ transitions. The included all-natural proof tracks these quantities and the unchanged registers.

The generator enumerates valid local parenthesizations and checks full outside-edge agreement. Eight original and eight lowered sources are retained. The displayed association is v00, with 211 distinct subtrees; v06 and v07 use different associations and attain 210. The allocator-compatible result below is the lowering of v00. The complete MP regressions cover all four residues, zero-pair aliases, both mismatch directions, and nonzero stacks; 5,688 cases pass for both the selected original and compatible lowered implementation. These finite tests corroborate, rather than replace, the loop proof.

Source	Primitives	Subtrees	Physical status
Accepted K215	4,751	202	Accepted 215 states
Best quotient splice	4,751	210	Unsupported leaf polarity in inherited allocator
Compatible lowering	4,753	212	Conservative price: 234 states

The first allocation failure is a missing right-child decrement entry contract, not a semantic counterexample. A compatible lowering inserts an operation clearing R_1 where that register is already zero. It repairs the interface at the measured cost. The allocator's identity control reproduces the original 215-state table exactly. The lowered source passes 4,752 internal-prefix and 4,753 primitive-entry checks. These are not a full arbitrary-register binary-runtime proof or a blank-start acceptance proof for a new 234-state consistency machine.

The conclusion is precise: quotient detachment opens a sound semantic design space, but this first implementation is larger. Primitive counts, distinct source subtrees, and physical states are different quantities. No 214-state result follows from the theorem or these tests.

7 The easiest additional 215-state proof

For a short incremental correctness proof, the existing conditional-marker CM215 is the best supported recommendation among the inspected alternatives. It preserves the calibrated source behavior and changes a small decoder region. Native roles 52 and 176 share a state; 176 becomes a predecessor-marker selector. Six rows change. The ordinary proof compares arbitrary initial tapes from control zero and preserves the complete tape and head at synchronized boundaries. Each parent step corresponds to one or three candidate steps [6].

A generic deterministic positive-simulation theorem supplies halting equivalence: initial states are related, terminal states agree, and every old nonterminal step is matched by a positive finite new block ending at a related state. Forward simulation transports old halting. If the old system never halted, repeated positive blocks would give arbitrarily long defined new runs, contradicting a finite new halt. Positive duration is essential for this reverse implication.

Incoming packets already contain checked CM215 transfer theorems and bridges to the repository's actual dense K215 semantics. Their small-build scopes are distinct from a full inherited ZFC endpoint replay. The recommendation is to integrate one exact adapter rather than re-prove the entire calculus, enumerator, runtime, and initialization for CM215.

The primary K215 remains the easiest machine to maintain without inherited assumptions beyond its own proof: it already has complete Lean and Rocq lanes. CM215 is attractive as an additional machine because its *marginal* proof is small. It is not simpler in absolute total dependency size merely because the transfer file is short.

M124 has another short all-tape marker proof, but the inspected incoming packet separates its standalone model from the full repository endpoint adapter. ZC215's zero-credit compiler is elegant compared with dirty-tail transfer schedules; it is not automatically simpler than primary K215, whose certified seed already has stack zero. Generalization-cache and GC-ZC variants alter more source semantics. Rebalanced addresses improve geometry but leave most logical obligations in place.

We prioritize the kind and number of new proof obligations. The alternative index also records rejected ideas: SB215 was refuted because its AX return reaches a selector rather than the scheduler. An older archive must not override that correction [7, 8].

8 The scoped search for 214 states

The physical packet surveys the primary table and twenty integrated alternative inputs. It uses a radius-five local-tape reachability overapproximation from all tapes at each input table's declared initial control and exact bounded whole-tape synchronization blocks. Primary K215 starts at control zero; ZC215 and GC-ZC215 start at control 74. Primary K215 gives 87,795 abstract control/window contexts. One grammar merges two noninitial states while allowing both retained rows to change and

leaving other providers fixed, with block length at most 31. A second grammar seeks two disjoint conditional mergers paid for by one synthesized selector.

No sufficient reduction was found. The baseline selector search recovers exactly the already known marker profiles 52/176, 124/127, and 160/190, which serve as controls. The added scope is the cross-candidate survey, not invention of the local-window method: earlier reports already study related families, sometimes with larger radii and larger enumerations.

A failed bounded proof-template check is not a proof that every proposed machine is wrong. The window abstraction can contain unreachable states; a valid join can exceed the bound; a transformation can require a shifted head, a different tape representation, or a phase invariant outside the grammar. These results do not establish a lower bound of 215 states.

The quotient rule attacks an independent limitation. It permits genuinely different inference behavior, so exact source-equivalence searches would reject it even when equiconsistency survives. Its first allocation is larger, but the semantic proof identifies an admissible design freedom. A joint search over arithmetic implementation, register placement, source sharing, and allocation is needed to turn that freedom into a smaller physical table.

9 Recommended integration order

1. Apply the pointwise-lifting patch, regenerate the exact Lean source, and rebuild the current typed endpoint clients. Retain source hashes and axiom reports.
2. Integrate one checked normalized source-certificate implementation from the incoming work. Preserve the public equivalence theorem and explicit equality witnesses for source data.
3. Add CM215 through its exact-table transport adapter, with the accepted K215 theorem as an explicit dependency.
4. Keep quotient detachment independent of machine acceptance. Preserve the singleton and pointwise counterexamples beside its semantic theorem.
5. Search jointly over the quotient implementation and allocator constraints. Restore a budget competitive with 215 before constructing a full new binary runtime proof.
6. For any smaller table, bind literal bytes, source, quantified primitive contracts, positive durations, and the complete blank terminal frame. A finite blank trace cannot establish the requested consistency equivalence.

This packet closes a concrete proof simplification and establishes a new semantic inference rule. It leaves a 214-state binary construction open.

10 Archive and reproduction

The archive contains this article and editable source, the reduced Lean proof and generator patch, the direct Coq companion, quotient semantic modules, sharp counterexamples, source variants, allocation and search receipts, finite checkers, and preserved upstream notices. Toolchains, proof objects, and full repository clones are excluded.

The top-level `verify.py` checks the manifest, key counts and file identities, the delivered proof bindings, and finite semantic regressions. A Python PASS is an executable audit, not a new kernel check. Component build drivers run the assistants separately and record exact versions and logs.

Integration notes identify exactly which sources and theorem interfaces were freshly compiled, and which binary-machine obligations remain outside this packet.

Build the PDF with two passes:

```
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

References

[1] ProveIt, accepted K215 and verification status, pinned revision 53269bcab1efc0f82d3022645bd7cf0c22d84959. https://github.com/VladimirReshetnikov/ProveIt/blob/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Research/ZFC/candidate-k215/README.md.

[2] ProveIt, Shorter checked K215 proofs. https://github.com/VladimirReshetnikov/ProveIt/tree/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification.

[3] ProveIt, DomRho8.lean. https://github.com/VladimirReshetnikov/ProveIt/blob/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/DomRho8.lean.

[4] ProveIt, Code254.lean: constructors, decoding, semantic laws. https://github.com/VladimirReshetnikov/ProveIt/blob/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/Code254.lean.

[5] ProveIt, EnumPerm.lean: command semantics and compiler. https://github.com/VladimirReshetnikov/ProveIt/blob/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/EnumPerm.lean.

[6] ProveIt, conditional-marker alternative. https://github.com/VladimirReshetnikov/ProveIt/tree/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/conditional-marker.

[7] ProveIt, materially distinct alternatives index. https://github.com/VladimirReshetnikov/ProveIt/blob/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/README.md.

[8] ProveIt, incoming research packets, 21 October 2026: K215 Proof Simplification, K215 Proof Transport, Structural Proof Simplification, ZFC215 Proof Simplification, and FM215 Proof Compression. Inspected as prior work and for exact build scope. https://github.com/VladimirReshetnikov/ProveIt/tree/53269bcab1efc0f82d3022645bd7cf0c22d84959/docs/incoming.